

Implement Client Script & UI Policy (Incident)

Project Documentation format

1. Introduction

- **Project Title:** Implement Client Script & UI Policy (Incident)
- **Team Members:** Mershitha T (Team Leader), Nimisha T (Team Member). Team ID: SWTID-2026-5468.

2. Project Overview

- **Purpose:** Demonstrate how ServiceNow client-side controls (UI Policies and Client Scripts) enforce data integrity on Incident records, so that incidents are created with complete and valid information.
- **Features:** UI Policy "High Impact Control" (Assignment group mandatory when Impact is High, Reverse if false); UI Policy Action making Urgency read-only; onChange script that sets Urgency to High; onSubmit script that blocks save when Assigned To is empty for High impact; onCellEdit script that blocks State changes from the list view.

3. Architecture

- **Frontend:** ServiceNow Platform UI: the Incident form and the Incident list view. The UI Policy and the client scripts run in the browser on these pages.
- **Backend:** The ServiceNow Now Platform instance in the cloud. No server-side code is written; the logic is configured through System UI → UI Policies and System UI → Client Scripts.
- **Database:** Incident records are stored in the incident table. Fields used: Impact, Urgency, Assigned To, Assignment group and State.

4. Setup Instructions

- **Prerequisites:** A ServiceNow instance and a user account with administrative or configuration access.
- **Installation:** No installation is needed. Create the UI Policy and the UI Policy Action, then create the three client scripts (onChange, onSubmit, onCellEdit) on the Incident table, following Tasks 1 to 5 of the project steps.

5. Folder Structure

- **Client:** Client-side configuration: UI Policy "High Impact Control" (with its Urgency action) and the client scripts "Auto set urgency for high impact" (onChange), "Prevent save if Assigned To missing" (onSubmit) and "Prevent state change via list edit" (onCellEdit).
- **Server:** Not applicable; the project has no server-side scripts. Records are stored in the incident table on the ServiceNow instance.

6. Running the Application

- Open the ServiceNow instance and use the Incident application:
 - **Form:** Incident → Create New (or open an existing incident) to test the UI Policy, onChange and onSubmit behaviour.
 - **List:** Incident → All to test the onCellEdit restriction on the State field.

7. API Documentation

- Not applicable. The project does not expose any REST endpoints; all logic runs on the Incident form and list through the UI Policy and Client Scripts.

8. Authentication

- Users sign in to the ServiceNow instance with their own account. Creating or changing the UI Policy and Client Scripts needs administrative or configuration access.
- Access to Incident records follows the standard ServiceNow roles; no tokens or custom authentication are added by this project.

9. User Interface

- Impact = High: Assignment group becomes mandatory and Urgency becomes read-only; Urgency is set to High with the message "Urgency set to High for High impact incident."
- Saving without Assigned To shows the error "Assigned To is mandatory for High impact incidents."
- Editing State in the list shows the alert "State cannot be updated using list editing. Please open the Incident."

10. Testing

- Test strategy: functional tests (FT-01 to FT-08), performance tests (PT-01 to PT-03) and user acceptance tests (TC-001 to TC-007) covering mandatory enforcement, successful save, the reverse condition, list edit blocking and form-based State update.
- Tools: the ServiceNow Incident form and list view in the browser.

11. Screenshots or Demo

- Insert screenshots of the Incident form and list for each test case here: [add screenshots / demo link].

12. Known Issues

- The controls run only in the user interface. Records created through other routes (for example imports or integrations) do not run client scripts.
- The onCellEdit script blocks every State change made from the list view, for all users.
- The onSubmit script checks only Impact = 1 (High) and Assigned To.

13. Future Enhancements

- Add a server-side check (for example a Business Rule) as a backup to the client-side rules.
- Extend the rules to more fields, conditions or tables.
- Allow selected roles to edit State from the list view.